

DEXAI Admin Portal

Application Reference & Knowledge Base

Internal document-processing / HITL review & client management portal — for the DEXAI Satori assistant

1. Overview

The DEXAI Admin Portal (a.k.a. TechDexAI Admin Portal, deployed at `manage.dexaitech.com`) is an internal web application for managing AI document processing. It lets staff review and correct extracted results (HITL — human-in-the-loop), audit every client's documents, track processing bugs, monitor infrastructure, manage users, and post announcements. It also hosts **DEXAI Satori**, an in-app AI assistant.

Stack: Next.js (App Router, React), PostgreSQL (single canonical database `MAIN_FINANCE_DB`), a separate FastAPI backend (`kdext_api`) for the customer API, RabbitMQ for events, and AWS S3 for document storage. Documents live across two S3 buckets: the current `kdext-finance-ai-document` (ap-southeast-2) and the legacy `receipt-hub-new` (eu-west-2); the viewer resolves a document by checking both.

2. Authentication & Sessions

- Staff/portal users are **internal_users**. They log in with email + password (bcrypt). A signed JWT is stored in the `auth_token` httpOnly cookie (24h).
- The JWT carries: `userId`, `email`, `roles[]`, `clientId` (for client-scoped accounts), and `pageAccess` (per-page permissions).
- Middleware verifies the token on every route, enforces role/route access, and injects `x-user-*` headers for API routes.
- The failed-login logout / rate-limiting is disabled — no account is locked out on repeated failures (failed attempts are still logged for audit).

3. Roles & Access Control

Access is role-based, with two additional layers: **client scoping** (a user only sees one client's data) and **per-page permissions** (toggles stored per user).

Role	Purpose	Can access
<code>SUPER_ADMIN</code>	Full administrator	Everything: all pages, all clients' data, user management, server monitoring, announcements authoring.

ADMIN / HITL	Staff reviewer	Home, Business Audit, HITL Edit, Bug Tracker (all clients). HITL reviewers default their client filter to <code>itadmin@capium.com</code> (changeable).
SERVER_MONITOR	Infra viewer	Only the Server Monitoring & Alerts pages. Read-only (cannot start/stop containers).
CLIENT	Client portal account	Dashboard, Business Audit, Bug Tracker — all scoped to their own client's data only . No HITL Edit, no Servers. Can create sub-users. Read-only (no Edit/Reprocess). Per-page toggles can further restrict which of these pages they see.
CLIENT_ADMIN / CLIENT_USER	Legacy client accounts	Scoped to their own client. CLIENT_ADMIN can manage CLIENT_USER sub-users.

Every signed-in role can also reach **DEXAI Satori** and **Announcements**.

Client scoping: an `internal_users.client_id` ties a client account to one row in the `users` table (the business/client). All Business-Audit / Bug-Tracker / document APIs force that scope server-side (`lib/clientAccess.js`) so a client can never read another client's data.

Per-page permissions: `internal_users.page_access` (JSON) holds `{dashboard, businessAudit, bugTracker}` booleans. A SUPER_ADMIN sets these when creating a client sub-user; middleware blocks toggled-off pages and the nav hides them. `null` = every page the role allows.

4. Pages & Features

Home / Dashboard (/)

Landing page with processing analytics. For client accounts it is scoped to their own client.

Business Audit (/dexai)

Directory of all companies and their users (from the main DB), with per-user document counts and done/total processing stats. Click a company to expand its users; click a user to open their results.

- **User results** (`/dexai/[userId]`): a searchable, filterable, paginated table of that user's documents — Result ID, HITL status, validation, HITL assignee, created date, bug status, document type, key environment, missing fields. Filters: type, status, validation, key environment, bug status, issue type. Staff can Edit (open in HITL Edit), assign a HITL reviewer, set bug status, export CSV, download documents. Client accounts get a read-only view (no Edit).
- **Result detail** (`/dexai/result/[requestId]`): document image + metadata + result. Tabs *Original Result* / *HITL Updated*, and a *JSON* / *Table* view toggle. JSON view shows the result JSON and the raw Processing Result side by side, full-height, always expanded. Includes a Reprocess control (staff only) and a Bug Tracking panel (issue type, description, bug status, comments).

HITL Edit (/missing-fields , "All Documents")

The human review queue — every processed document with its HITL status, validation, assignee, bug status, issue type, and missing fields. Reviewers open a document, correct the extracted fields, and publish.

Filters + search (search is global, ignoring the client filter). HITL reviewers/admins default to `itadmin@capium.com`'s data, changeable. Not visible to CLIENT accounts.

HITL result editing & validation: each document has an immutable `formatted_result` (original extraction) and an editable `hitl_updated_result` (human-corrected copy). Publishing an edit sets `validation = true` and `status/hitl_status = COMPLETED`. The viewer's "HITL Updated" tab shows `hitl_updated_result`; "Original Result" shows `formatted_result`.

Bug Tracker (`/bug-tracker`)

Every document that has an issue logged against it, across all companies (or scoped, for clients). Columns: **Bug ID** (a stable human-friendly id like `BUG-00918`), Result ID, HITL assignee, bug status, client email, document type, bug-created date, comments.

- **Issue types:** wrong document type; table items are wrong; net amount and tax wrongly calculated; supplier name not extracted; single input page multiple output; credit-debit value swapping; bank account name/number wrongly extracted; opening/closing date wrongly extracted; opening/closing balance wrongly extracted; missing transaction data; quantities not extracted; tax amount not extracted; low quality document input; handwritten input document; receipt number wrongly extracted; line level discount wrongly extracted; connection error; no issue; known issue; other.
- **Bug statuses:** Open, TO_BE_TESTED, Closed (or none).
- Per-document **comments** (email + message + timestamp), CSV export, document download, bulk status updates. Search matches request/result/transaction ids, doc type, client, and the Bug ID.

Server Monitoring (`/server-monitor`) & Alerts (`/alerts`)

Live view of the backend servers — proxies each server's monitoring agent (Flask app on port 5000). Shows CPU / memory / disk / GPU (with progress bars), the Docker container list (running/stopped), and full container logs with auto-refresh. Admins can start/stop/restart/remove containers (confirmation on stop/restart/remove). SERVER_MONITOR accounts get a read-only version. Alerts is a feed of monitoring errors/critical events (stored in `monitor_alerts`) with resolve actions.

Announcements (`/announcements`)

Team announcement feed visible to **everyone**. Only SUPER_ADMIN can post (title, rich text, optional PDF/image attachments, optional date/time defaulting to now, and a status). Each announcement carries a status shown as a glowing-dot pill: **Active** (blue), **Resolved** (green), **Maintenance** (amber), **Release Note** (purple), **Documentation** (teal). Cards are collapsed to the title; clicking expands the body and attachments. A super admin can change any announcement's status inline.

DEXAI Satori (`/satori`)

An in-app, ChatGPT-style AI assistant available to all users. Has a conversation history sidebar, prebuilt starter prompts, and a chat thread with a typing indicator. Answers questions about documents, results, and workflows. (Backend model endpoint is pluggable.)

User Logs / Team Members / Clients (/user-logs)

SUPER_ADMIN sees three tabs: **Team Members** (internal staff), **Clients** (business users, their portal logins, and nested sub-users), and **HITL Workload**. From the Clients tab a super admin can **Add a sub-user** to any client and tick which pages that sub-user can see (Dashboard / Business Audit / Bug Tracker). CLIENT / CLIENT_ADMIN accounts get a restricted version to manage their own sub-users.

Document viewer (/view/[id])

Standalone document view — file preview (resolved across both S3 buckets), Original vs HITL-Updated result tabs, edit history/audit, and reprocess.

5. Core Concepts

- **Document processing lifecycle:** a request (request_id , e.g. req_...) is submitted, processed, and produces a result (result_id , e.g. PDR-136578). Status flows through processing → COMPLETED / TO_BE_TESTED. validation=true means it passed mandatory-field validation (or a reviewer finalized it).
- **HITL workflow:** documents needing review are TO_BE_TESTED; a reviewer is assigned, edits hitl_updated_result , and publishes → COMPLETED + validation true.
- **Bug tracking:** a document gets an issue_type (which puts it in the Bug Tracker and assigns a Bug ID), an optional description, a bug status, and threaded comments.
- **Client scoping & sub-users:** client accounts are locked to one client's data; a client (or a super admin on their behalf) can create sub-users that inherit the same scoped, read-only view.
- **API keys:** business users (the users table) each have production / sandbox / test API keys. Keys authenticate on the FastAPI backend and are gated by keys_active (admin approval, with a 15-day grant that auto-expires and must be renewed).

6. Data Model (key tables)

Table	What it holds
document_processing_requests	The core table — one row per document. Key columns: result_id , request_id , user_id , ocr_document_type , document_path , formatted_result (original JSON), hitl_updated_result (edited JSON), processing_result (raw), status , hitl_status , validation , hitl_assigned_to , issue_type , issue_description , bug_status , bug_tracker_id , bug_flagged_at , comments (JSON array), submitted_at , completed_at .
internal_users	Staff + client portal accounts. Columns include email , password_hash , is_active , client_id (client scope), page_access (per-page toggles), failed_login_attempts , locked_until .
users	Business/customer accounts. Includes API keys (api_key , sandbox_api_key , test_api_key), keys_active ,

	<code>keys_activated_at</code> , <code>company_name</code> .
<code>roles</code> , <code>user_roles</code>	Roles and the <code>internal_user</code> ↔ role mapping.
<code>announcements</code> , <code>announcement_attachments</code>	Announcement posts (title, body, status, <code>announced_at</code> , author) and their binary PDF/image attachments.
<code>monitor_alerts</code>	Server-monitoring alerts/errors (severity, category, server, container, message, first/last seen, resolved).
<code>api_logs</code>	API call audit log (user, endpoint, success, status code, latency, error).
<code>clients</code> , <code>businesses</code>	Client and business records.
<code>webhooks</code> , <code>webhook_deliveries</code>	Customer webhook registrations and delivery attempts (e.g. <code>document.corrected</code> on HITL publish).
Other tables: <code>bank_transaction_scores</code> , <code>anomaly_models</code> , <code>bank_anomaly_models</code> , <code>dexai_invoices</code> , <code>dexai_audit_log</code> , <code>document_types</code> , <code>document_processing_logs</code> , <code>internal_activity_logs</code> , <code>internal_testing_requests</code> , <code>email_verification_tokens</code> , <code>otp_verifications</code> , <code>rate_limit_tracking</code> , <code>api_endpoints</code> .	

7. API Endpoints (reference)

Group	Endpoints
Auth	<code>POST /api/auth/login</code> , <code>POST /api/auth/logout</code> , <code>GET /api/auth/me</code> , <code>POST /api/auth/change-password</code>
Business Audit	<code>GET /api/dexai/overview</code> , <code>/api/dexai/users</code> , <code>/api/dexai/users/[userId]</code> , <code>/api/dexai/users/[userId]/results</code> (+ <code>/export</code> , <code>/download-documents</code>), <code>/api/dexai/result/[requestId]</code> , <code>/api/dexai/businesses/[businessId]</code> , <code>/api/dexai/clients/[clientId]</code>
HITL / Missing fields	<code>GET /api/missing-fields</code> (+ <code>/export</code> , <code>/download-documents</code>), <code>/api/hitl-users</code> , <code>/api/hitl-stats</code> , <code>/api/filter-options</code>
Documents	<code>GET /api/document/[id]</code> , <code>/download</code> , <code>POST .../update</code> , <code>.../update-status</code> , <code>.../update-bug-tracking</code> , <code>.../assign-hitl</code> , <code>.../reprocess</code> , <code>.../comments</code> (+ <code>/[commentId]</code>)
Bug Tracker	<code>GET /api/bug-tracker</code> (+ <code>/stats</code> , <code>/export</code> , <code>/download-documents</code>)
Announcements	<code>GET,POST /api/announcements</code> , <code>PATCH,DELETE /api/announcements/[id]</code> , <code>GET /api/announcements/attachments/[id]</code>
Server Monitoring	<code>GET /api/server-monitor/dashboard</code> , <code>/logs</code> , <code>POST /api/server-monitor/action</code> ; Alerts: <code>GET /api/alerts</code> , <code>/summary</code> , <code>POST /api/alerts/[id]/resolve</code>
User management	<code>GET /api/user-logs</code> (+ <code>/[userId]</code> , <code>/credentials</code> , <code>/provision-all</code>), <code>GET,POST /api/client-users</code> (+ <code>/all</code> , <code>/[internalUserId]</code> , <code>/credentials</code>), <code>GET /api/team-members</code> (+ <code>/[id]</code> , <code>/credentials</code>)

Other	GET /api/documents , /api/transactions (+ /[resultId]), /api/health , /api/satori/chat (planned)
-------	---

8. How the UI Works — Navigation & Layout

Every page shares a fixed top **Navbar**:

- **Brand** (left): "TechDexAI / Admin Portal".
- **Nav links** (center) — shown by role: *Home*, *Business Audit*, *HITL EDIT*, *Bug Tracker* (staff); *User Logs* (super admin & client admins); *Servers + Alerts* (super admin & SERVER_MONITOR); and *DEXAI Satori + Announcements* (everyone). A CLIENT account sees only the pages its per-page toggles allow.
- **Theme toggle** (right): sun/moon switch between light and dark; the choice is persisted. All colors come from CSS variables so both themes stay consistent.
- **User menu** (right): shows the signed-in email, with *Change password* and *Logout*.
- **Alerts badge**: a red/amber count on the Alerts link (red = active critical, amber = active warnings), polled every 30s.

Data loading & freshness: pages fetch through React Query — showing loading placeholders, caching results, and auto-refetching (e.g. the HITL queue refreshes ~every 20s, the alerts badge ~30s, container logs ~5s). Actions give instant feedback via toast notifications ("Published", "Sub-user added", errors, etc.), and many saves update the on-screen data optimistically without a full reload.

9. Visual Language & Conventions

The UI uses consistent color-coded pills and dots so state is readable at a glance:

Element	Meaning
HITL Status: Completed / TO_BE_TESTED / Pending	Where a document sits in the review pipeline.
Validation: Valid / To be tested	Green dot = passed validation / finalized; red = still needs review.
Bug Status: Open / TO_BE_TESTED / Closed	Lifecycle of a logged issue. Editable inline via a dropdown.
Key Environment: Sandbox / Testing / Production	Which API key environment produced the document.
Announcement status: glowing dot — Active (blue), Resolved (green), Maintenance (amber), Release Note (purple), Documentation (teal)	Category/state of an announcement.
Result ID (e.g. PDR-136578) shown in accent color with the req_... request ID beneath in muted mono; Bug ID as BUG-00918 .	Primary identifiers per row.

Tables everywhere share the same behavior: click a column header to sort (ascending/descending arrows), a search box (debounced ~350ms) above, dropdown filters that combine with AND, row checkboxes for bulk

actions, and paged results with Prev/Next. On the Bug Tracker and HITL pages a *search is global* — it ignores the client filter so a Bug ID or Result ID is always findable.

10. Common Workflows (step by step)

Review & correct a document (HITL)

1. Open *HITL EDIT* (or click **Edit** on a document in Business Audit). The list defaults to `itadmin@capium.com`'s data for staff — change the client filter to widen it.
2. Open a `TO_BE_TESTED` document. The viewer shows the file on the left and two tabs: *Original Result* (read-only) and *HITL Updated* (editable).
3. Edit fields in the HITL Updated tab (scalars, nested objects, and line-item tables; multi-receipt documents have a per-receipt pager).
4. Click **Publish**. This saves `hitl_updated_result`, sets validation = true and status = COMPLETED, appends an audit entry (who/when/which fields), and fires a `document.corrected` webhook to the customer.

Log & triage a bug

1. On a result's Bug Tracking panel (or the Bug Tracker row), pick an **Issue Type** — this assigns a permanent Bug ID and lists the document in the Bug Tracker.
2. Add an optional description, set a **Bug Status** (Open / `TO_BE_TESTED` / Closed), and add threaded comments.
3. Filter/search the Bug Tracker by status, issue type, client, or Bug ID; export CSV or download the source documents.

Assign / reprocess

- **Assign**: use the HITL dropdown on a row to assign a reviewer.
- **Reprocess**: on a result detail, pick the document type and click *Reprocess* to re-run the pipeline in place (keeps the same `request_id`). Staff only — hidden for client accounts.

Add a client sub-user with page access (super admin)

1. Go to *User Logs* → *Clients*, find the client, click **Add sub-user**.
2. Enter email / name / password and tick which pages they can see: Dashboard, Business Audit, Bug Tracker.
3. The sub-user logs in scoped to that client only, read-only, seeing exactly the ticked pages.

Activate a client's API keys

In the customer portal / admin, a client has production / sandbox / test keys shown as Active or Not Active. An admin activation flips `keys_active` on (a 15-day grant); after it expires it reverts to "pending approval" and must be renewed.

Post an announcement (super admin)

1. Open *Announcements*. Fill the title + body, optionally attach PDFs/images, pick a status, and set a date/time (blank = now).
2. Click **Post**. Everyone sees it in the feed — collapsed to the title + status dot; clicking expands the body and attachments. A super admin can change any announcement's status inline.

Use DEXAI Satori

1. Open *DEXAI Satori*. Click a prebuilt prompt or type a question (Enter to send, Shift+Enter for a newline).
2. Messages appear as a chat thread with a typing indicator; conversations are saved to the History sidebar (reopen or delete them). "New chat" starts a fresh thread.

Monitor servers

1. Open *Servers*, switch between Server 1 / Server 2. See CPU / memory / disk / GPU meters and the container list.
2. Click a container to view its full logs (auto-refreshing every 5s). Admins can start / stop / restart / remove (with confirmation); SERVER_MONITOR accounts are read-only.
3. *Alerts* lists monitoring errors/criticals; resolve them or clear the feed.

11. Technical UI Notes

- **Framework:** Next.js App Router, React client components, React Query for server state, inline styles driven by CSS variables (theme-aware), lucide-react icons, react-hot-toast for notifications.
- **Access enforcement is layered:** middleware redirects disallowed routes; the Navbar hides links a role/permission can't use; and every data API re-checks the role and forces the client scope server-side (never trusting client-supplied params).
- **Result viewer:** Original vs HITL tabs × JSON vs Table view. JSON view shows the result and the raw processing payload side by side, full-height and always expanded; Table view renders the same data as labelled fields and line-item tables.
- **Satori** stores conversation history in the browser's localStorage; its model endpoint is pluggable (a single request function) and currently returns preview responses.
- **Announcement attachments** are stored in the database and streamed back through an API endpoint (images render inline, PDFs open in a new tab).
- **Document files** are fetched from S3 with short-lived signed URLs; the resolver checks both the current and legacy buckets so a file is found regardless of which bucket holds it.